

Inheritance

Mladen Ivkovic

**COMP2221 Programming Paradigms –
Object-oriented Programming**

Join by
Web

PollEv.com
/mladenivkovic356



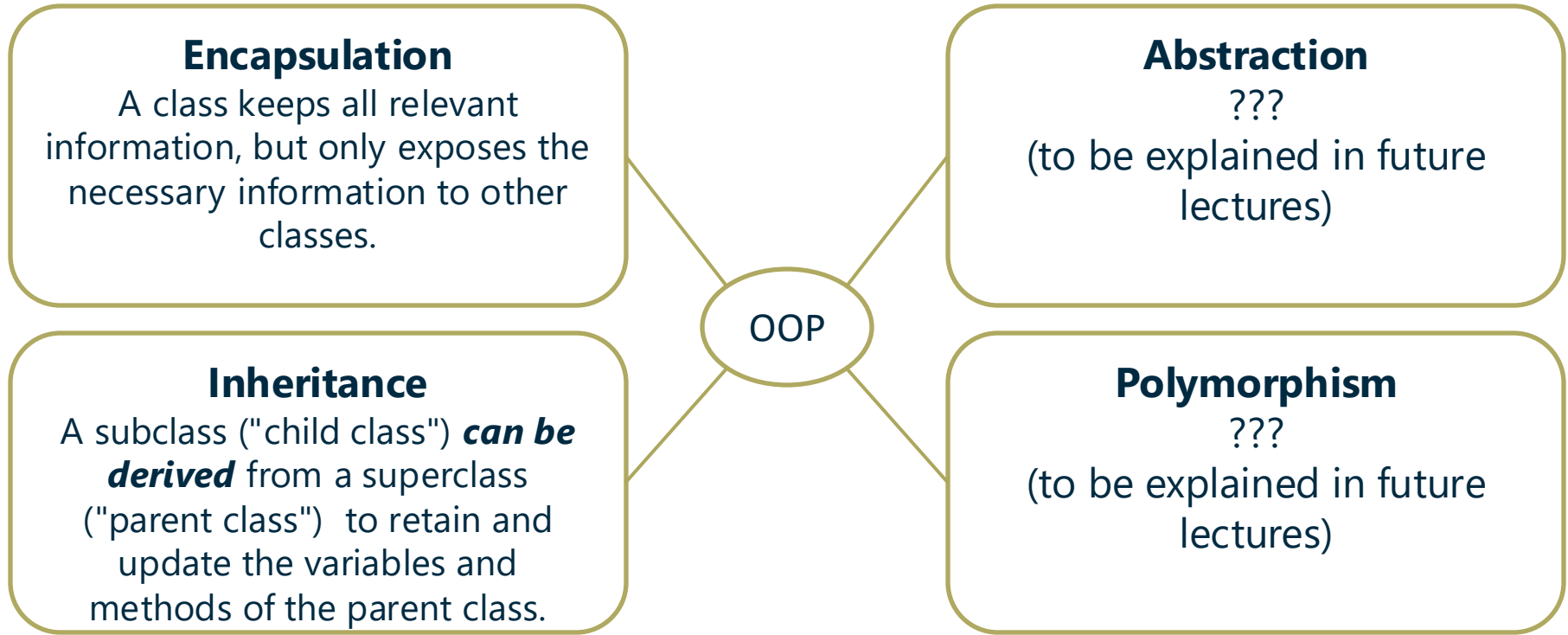
What Will You Learn in this Lecture?

- What is inheritance?
- How to code with inheritance?
- Abstract Classes vs Interfaces
- What is the UML class diagram?



Inheritance Design

The Four Principles of OOP

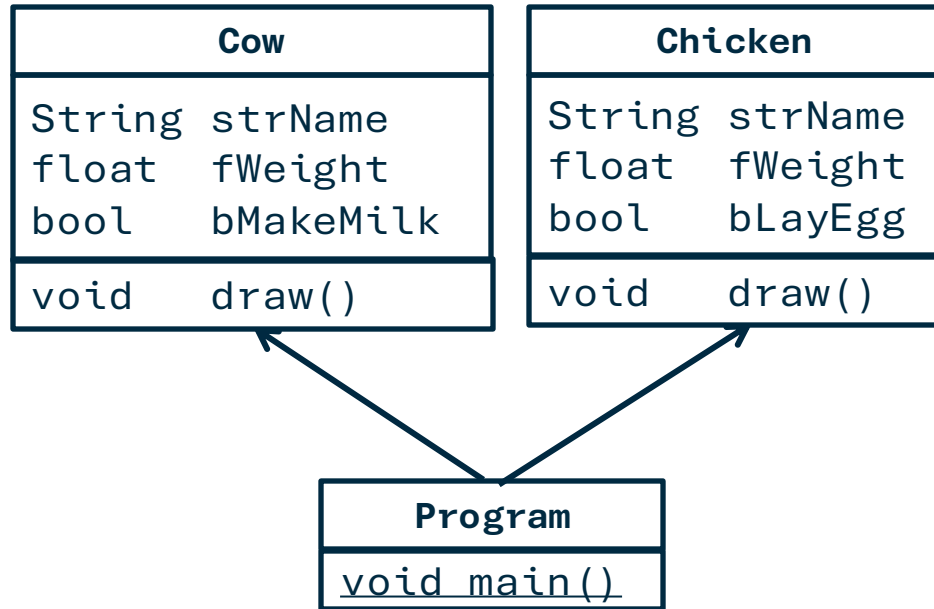


An Example to Explain Inheritance – Required Variables

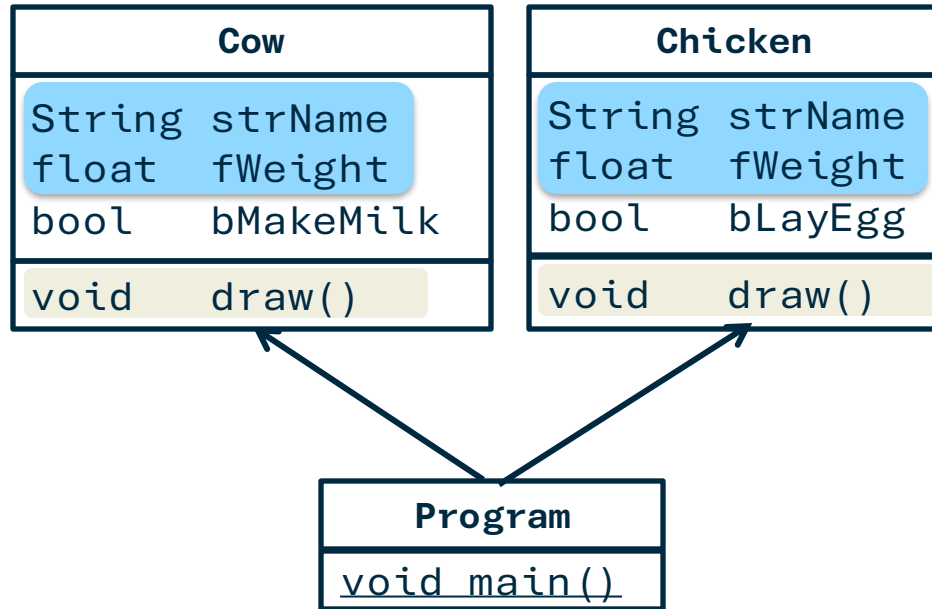
- An animal farming game
- The Cow class
 - Name (String)
 - Weight (float)
 - Produce Milk (bool)
- The Chicken class
 - Name (String)
 - Weight (float)
 - Lay Egg (bool)



An Example to Explain Inheritance – UML Class Diagram

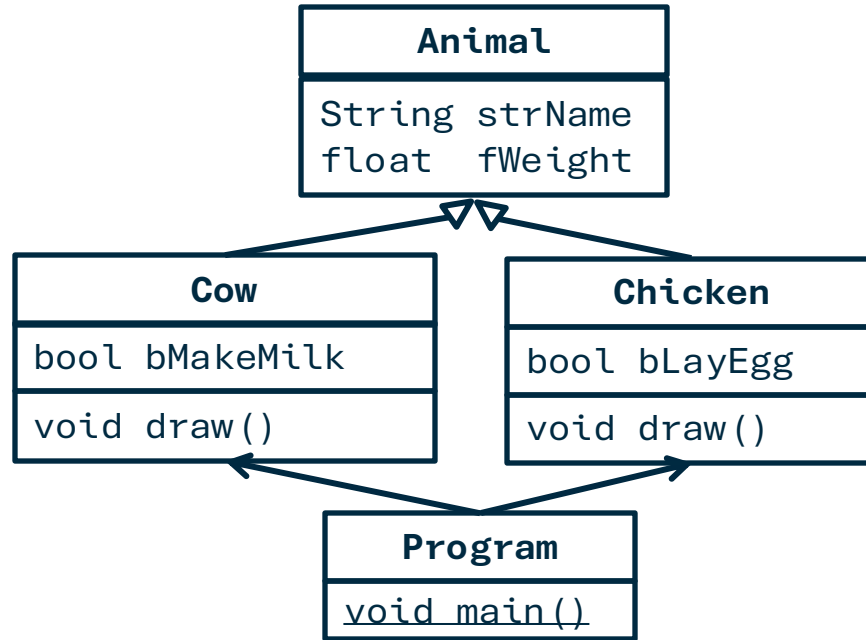


An Example to Explain Inheritance – UML Class Diagram

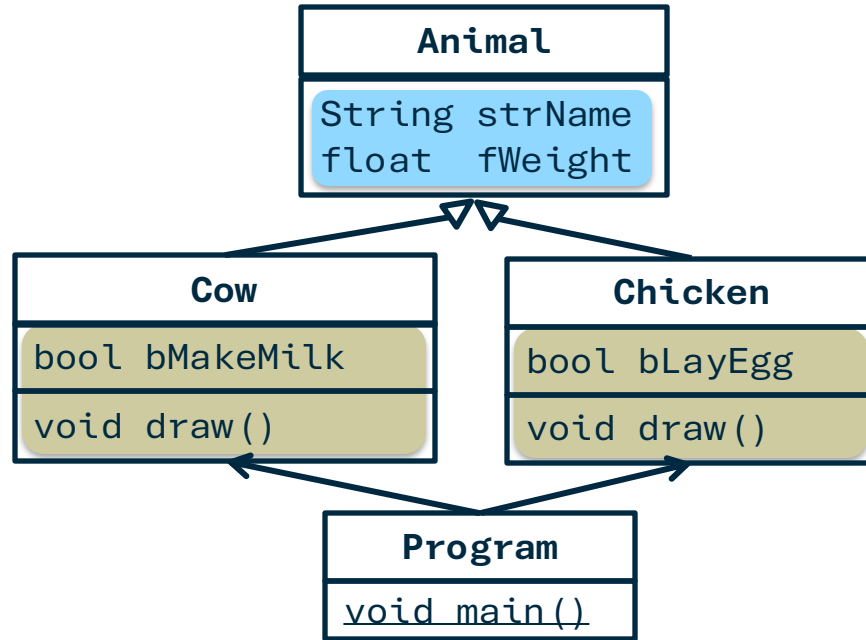


- Same name
- Same "nature" (type)
- Same name
(the function header in this case)
- Representing same behaviour
- But: different implementations

An Example to Explain Inheritance – UML Class Diagram with Inheritance



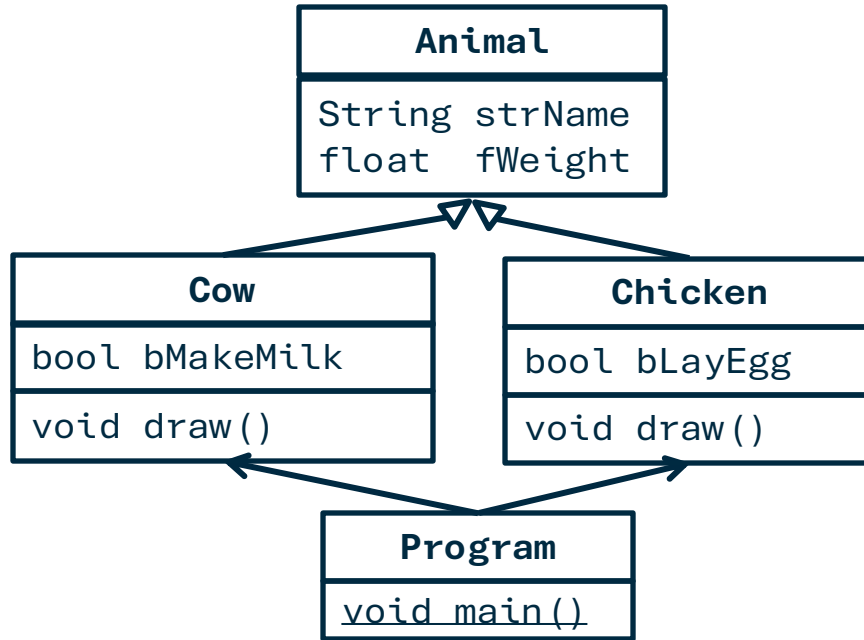
An Example to Explain Inheritance – UML Class Diagram with Inheritance



Both the Cow and Chicken classes retain (i.e. inherit) `strName` and `fWeight` from the Animal class

The Cow and Chicken classes implement their class-specific variables and methods

Quick Question



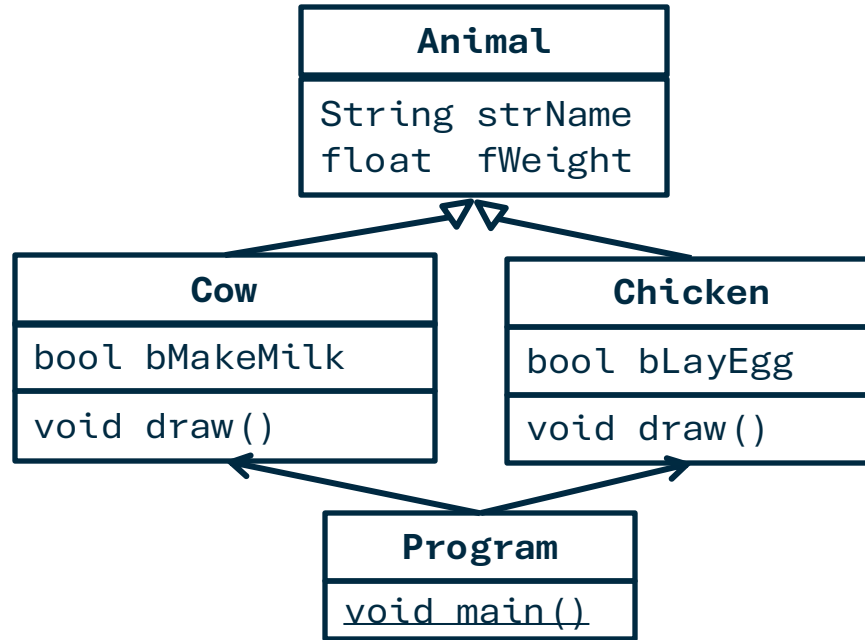
Assuming that the `main()` function has the following local variables:

```
Cow cow1 = new Cow(...);
Cow cow2 = new Cow(...);
Chicken chicken1 = new Chicken(...);
Chicken chicken2 = new Chicken(...);
```

Which of the following are invalid?

1. `cow1.strName`
2. `cow2.fWeight`
3. `cow1.bMakeMilk`
4. `cow2.bLayEgg`
5. `cow1.draw()`
6. `chicken1.strName`
7. `chicken2.fWeight`
8. `chicken1.bMakeMilk`
9. `chicken1.bLayEgg`
10. `chicken2.draw()`

Quick Question



Assuming that the `main()` function has the following local variables:

```
Cow cow1 = new Cow(...);
Cow cow2 = new Cow(...);
Chicken chicken1 = new Chicken(...);
Chicken chicken2 = new Chicken(...);
```

Which of the following are invalid?

1. `cow1.strName`
2. `cow2.fWeight`
3. `cow1.bMakeMilk`
4. `cow2.bLayEgg`
5. `cow1.draw()`
6. `chicken1.strName`
7. `chicken2.fWeight`
8. `chicken1.bMakeMilk`
9. `chicken1.bLayEgg`
10. `chicken2.draw()`

Inheritance Coding

Syntax of Inheritance

```
class SuperClass {  
    // Parent properties and methods  
}  
  
class DerivedClass extends SuperClass {  
    // Additional properties and methods  
}
```

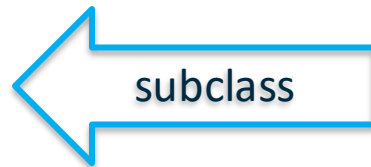
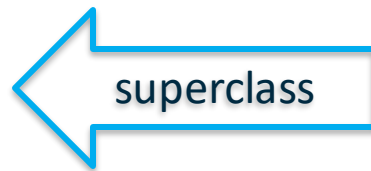
- The **extends** keyword is used for inheritance.
- The subclass **inherits non-private members** of the superclass.
- The private modifier specifies that the member can only be accessed in its own class

Superclasses & Subclasses

- **Superclass (Parent Class):** The class whose properties are inherited.
- **Subclass (Child Class):** The class that inherits from the superclass.

Example:

```
class Vehicle {  
    String brand = "Ford";  
}  
  
class Car extends Vehicle {  
    String model = "Mustang";  
}
```



Method Overriding

- Sometimes, we want to overwrite a superclass' method in a subclass.
- Example: we want to adapt it for the subclass' purposes
- How do we achieve that?

```
class Animal {  
    void makeSound() {  
        // does something  
    }  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        // does something else  
    }  
}
```


Method Overriding

- **Method Overriding:** Redefining a method in the subclass that exists in the superclass.
- Uses `@Override` annotation.

Example:

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

Method Overloading

- **Recap: Method Overloading:** Same method name, different signatures (parameters).
- Happens *within* the same class.
- **Example:**

```
class MathUtils {  
    int add(int a, int b) {  
        return a + b;  
    }  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

@Override Annotation

- **Note:** Using `@Override` is not *strictly* necessary.
- Both of these examples are valid Java code that do the same!

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

@Override Annotation

- **Note:** Using `@Override` is not strictly necessary.
- So why use it?
- It's good practice!
 - Tells you/other developers that method is overridden
 - Compiler will throw error if you're trying to override a method that isn't overridable
 - Example: You have a typo in the method name or parameter list
 - Example: You change method name/parameter list in superclass → compiler will tell you about all subclasses that need adapting!
 - Otherwise: Compiler will assume you're overloading a method.

Recap: `this` keyword

- Used to refer to the **current object** in a method or constructor.
- Common usage: eliminate confusion between class attributes and parameters with the same name

```
class Car {  
    private String brand;  
    private int speed;  
  
    // Constructor with validation  
    public Car(String brand) {  
        this.brand = brand;  
    }  
}
```

The `super` Keyword

- **super** refers to the parent class.
- Used to access parent class methods and constructors.

Example:

```
// Start of code
class Animal {
    Animal() {
        System.out.println("Animal constructor called");
    }
}

class Dog extends Animal {
    Dog() {
        super(); // Calls the constructor of Animal
        System.out.println("Dog constructor called");
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
//End of code
```

Output:

Animal constructor called
Dog constructor called

The `super` Keyword

- Another example: Bypassing overrides

```
// Start of code
class Animal {
    void makeSound() {
        System.out.println("Animal sound!");
    }
}

class Cat extends Animal {
    @Override void makeSound(){
        super.makeSound();
        System.out.println("Meow.");
    }
}
```

```
public class MyMain {
    public static void main(String[] args) {
        Cat myCat = new Cat();
        myCat.makeSound();
    }
}
// End of code
```

Output:

```
Animal sound!
Meow.
```

Access Modifiers in Inheritance

- New access modifier: **protected**
- Variable/method accessible:
 - In same package
 - In the class where it is declared
 - In subclasses which inherit from class
 - Not outside class/subclass/package.

Access Modifiers in Inheritance

	default	private	protected	public
Same class	✓	✓	✓	✓
Same package subclass	✓	✗	✓	✓
Same package non-subclass	✓	✗	✓	✓
Different package subclass	✗	✗	✓	✓
Different package non-subclass	✗	✗	✗	✓

- **default/no modifier:** Accessible in same package.
- **private:** Accessible only within declared class.
- **protected:** Accessible in same package and subclasses.
- **public:** Accessible for all classes.

Preview:

Abstract Classes and Interfaces

Abstract Classes

- Contain ***abstract methods*** (no implementation).
- Concrete subclasses ***must*** implement those methods.
 - Concrete = non-abstract subclasses; subclasses you want to make objects of

```
abstract class Animal {  
    abstract void makeSound();  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

Abstract Classes

- Cannot be instantiated.

```
abstract class Animal {  
    abstract void makeSound();  
}  
  
public class MyMain {  
    Animal a = new Animal();  
}
```

Error: Cannot instantiate the type Animal

What are Interfaces?

- Interfaces define a **contract** that classes must follow.
- ***All methods*** in interfaces are abstract (before Java 8).
 - After Java 8: Static methods allowed too.
- Define **behaviours** that a class must implement.
- Supports **multiple inheritance**.
- **Example:**

```
interface Vehicle {  
    void start();  
    void stop();  
}
```

Interfaces: Example

```
interface Animal {  
    void makeSound();  
}  
  
class Dog implements Animal {  
    public void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

- Comparing Abstract classes vs Interfaces: Next lecture!

Interface Example (as multi-inheritance)

```
interface ElectricVehicle {
    void chargeBattery();
}

interface Vehicle {
    void start();
    void stop();
}

class ElectricCar implements Vehicle, ElectricVehicle {
    public void start() {
        System.out.println("Electric car starts silently.");
    }
    public void stop() {
        System.out.println("Electric car is stopping.");
    }
    public void chargeBattery() {
        System.out.println("Charging battery...");
    }
}
```

Summary so far

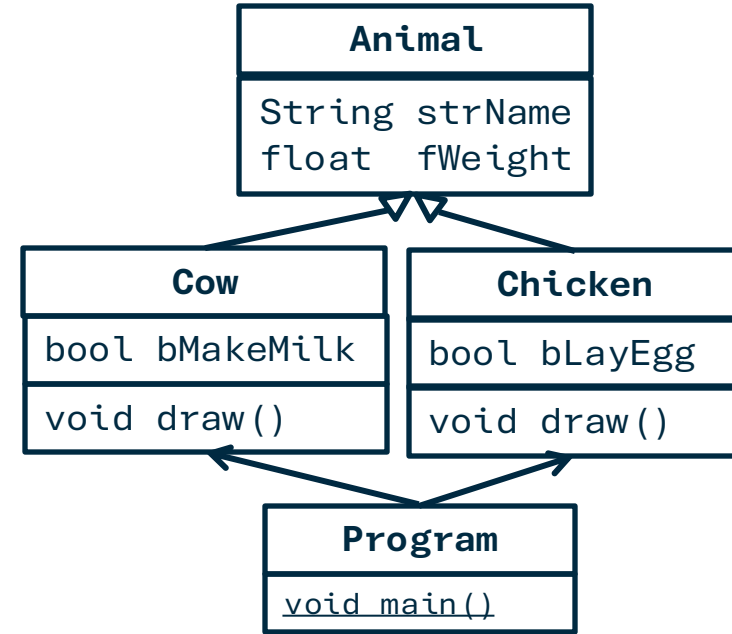
- Inheritance allows **code reuse** and **hierarchical relationships**.
- Inheritance can be provided via (abstract) classes or interfaces.
- Abstract classes & interfaces define ***contracts*** for subclasses.
- **Overriding** enables dynamic behaviour.
- The ``super`` keyword allows access to parent class members.

UML Class Diagram

The Unified Modeling Language (UML)

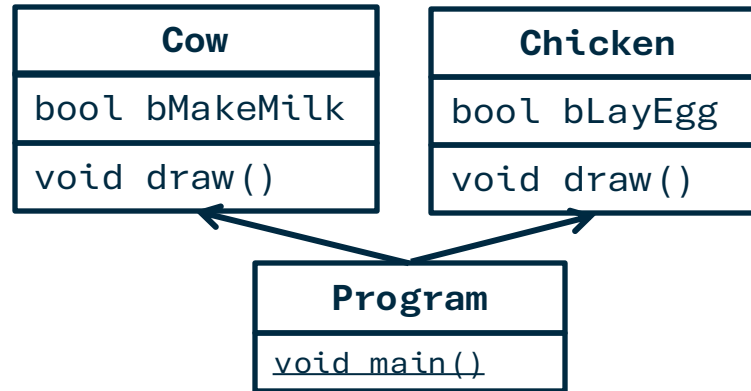
Class Diagram

- In UML, different arrows have different meanings
 - It is important to use the right arrows, or your design may be misunderstood
- We will introduce a few arrows today, with more in the future



UML Arrows – Association

- (Directed) Association 
 - (Directed) relationship: One needs the other, but not the other way around
 - A class (container) creates an object of another class (content)



UML Arrows – Composition



- A more specific expression of association
- "Is composed of" relation:
 - part class cannot exist independently of the whole class
 - A class (container) creates an object of another class (content), and the content is destroyed when the container is destroyed
 - e.g. a "Car" class has an object of the class "Engine"

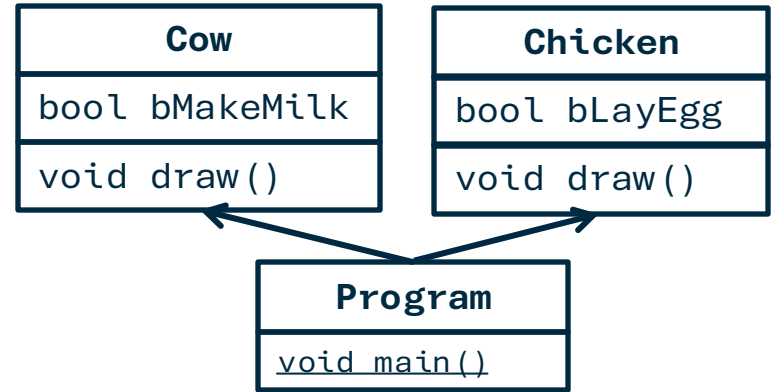
UML Arrows – Aggregation



- A more specific expression of association
- "Contains/Collects/Aggregates" relation:
 - A class (*container*) creates an object of another class (*content*), and the content is NOT destroyed when the container is destroyed.
 - e.g. a "Professor" class has multiple objects of the class "Course" – when the professor stops teaching a course, the course object will still exist but will be assigned to another professor object.

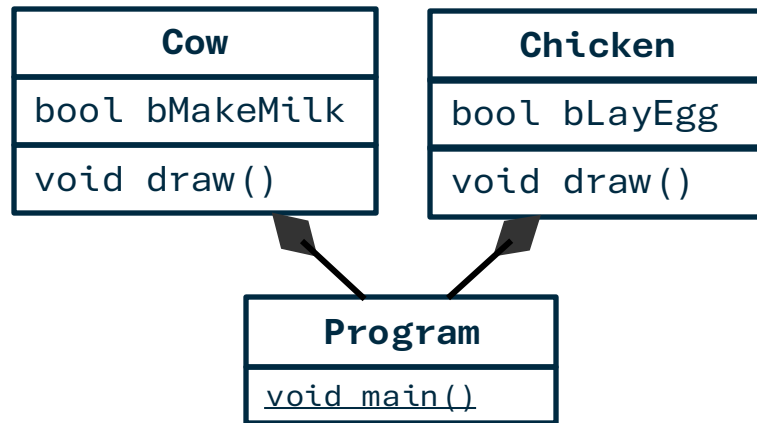
Composition and Aggregation – Which One To Use?

- You should use composition and aggregation instead of association where appropriate
- Quick question: Which one to use for the example here?



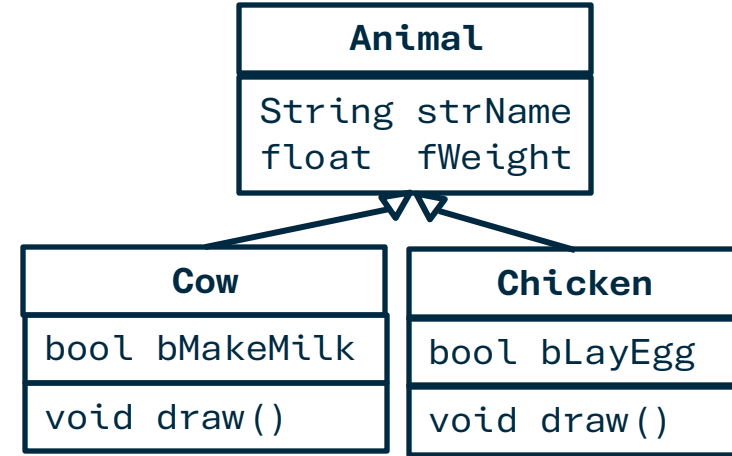
Composition and Aggregation – Which One To Use?

- You should use composition and aggregation instead of association where appropriate
- Quick question: Which one to use for the example here?
- **Composition!**



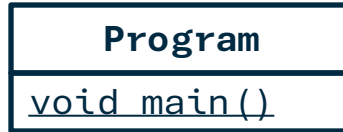
UML Arrows – Inheritance

- Inheritance 
 - A child class inherits a parent class



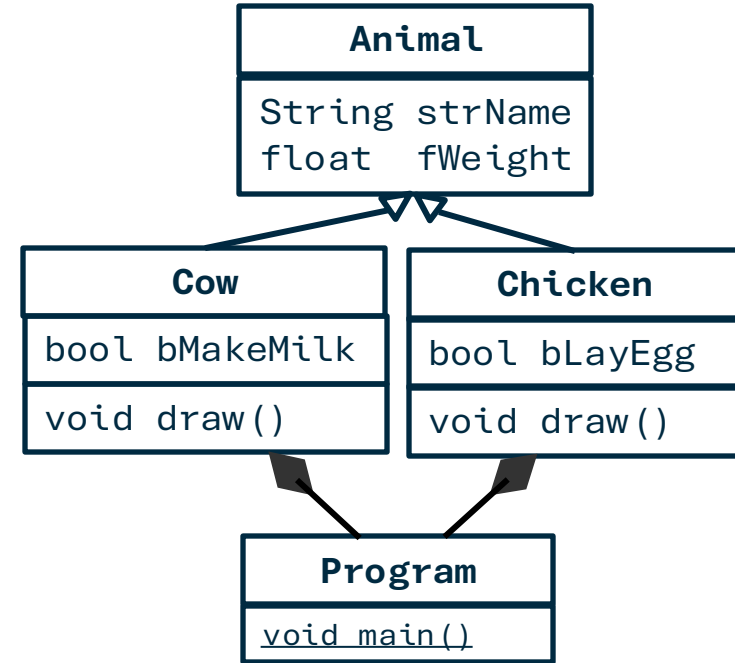
UML for static

- Underline to denote static methods and variables



Finally on UML...

- You should be able to explain a system designs by reading the UML
- Note
 - There are different UML symbol schemes
 - What software to use for drawing UML?



What software to use for drawing UML?

- Microsoft Visio,
- Lucidchart,
- draw.io (also known as diagrams.net),
- PlantUML (for text-based UML),
- Visual Paradigm, or
- Modelio

Summary

- Inheritance
 - Design
 - Coding
- UML class diagrams



The End

Any Questions?



Exercises

- Either tell me what the output of the code is
- Or tell me why the code is wrong and how to fix it

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:" + brand);  
    }  
}  
  
class Ferrari extends Car {  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

Output:
Brand:Unknown Brand

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    String brand = "Ferrari";  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {
    String brand = "Unknown Brand";
    void printInfo() {
        System.out.println("Brand:" + brand);
    }
}

class Ferrari extends Car {
    String brand = "Ferrari";
    // RE-DECLARATION OF STRING BRAND!
}
```

```
public class MyMain {
    public static void main(String[] args) {

        Ferrari F80 = new Ferrari();
        F80.printInfo();
    }
}
```

Output:
Brand:Unknown Brand

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    brand = "Ferrari";  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    brand = "Ferrari";  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

Output:

Doesn't compile – can't do assignments on class level

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    Ferrari(){  
        this.brand = "Ferrari";  
        super.printInfo();  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    Ferrari(){  
        this.brand = "Ferrari";  
        super.printInfo();  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

Output:
Brand:Ferrari
Brand:Ferrari

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    @Override void printinfo(){  
        System.out.println("This is a Ferrari");  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {  
    String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    @Override void printinfo(){  
        System.out.println("This is a Ferrari");  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

Output:

Doesn't compile - no method
`printinfo` to override

What is the output of this code?

```
class Car {  
    protected String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    @Override void printInfo(){  
        System.out.println("This is a Ferrari");  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {  
    protected String brand = "Unknown Brand";  
    void printInfo() {  
        System.out.println("Brand:"+brand);  
    }  
}  
  
class Ferrari extends Car {  
    @Override void printInfo(){  
        System.out.println("This is a Ferrari");  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Ferrari F80 = new Ferrari();  
        F80.printInfo();  
    }  
}
```

Output:
Brand:Ferrari